INTERNAL ENGINEERING DOCUMENTATION

Detailed Architecture

AI Document Auditor — component-level reference: every layer, every endpoint, every data model, and the specific production issues that shaped the current design.

Companion to: AI-Document-Auditor-Project-Overview.pdf

Prepared for: Harrison Ekpobimi

Date: September 2026

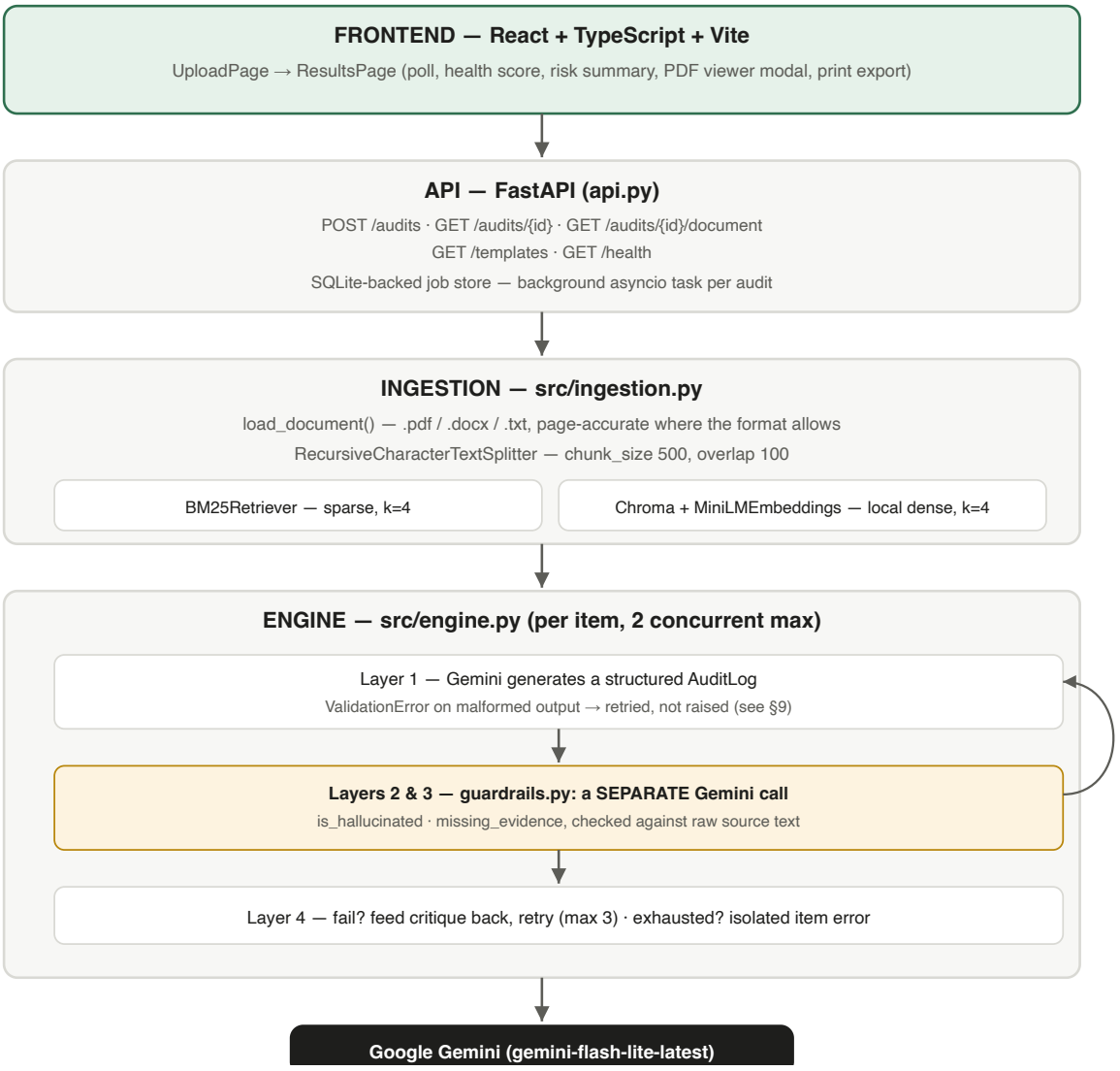
Audience: Engineers working on or extending this codebase

Contents

1. System Diagram
2. Frontend Architecture
3. API Layer & Endpoint Reference
4. Ingestion Layer
5. Engine Layer — The Audit Loop
6. Guardrails Layer
7. Data Models
8. Request Lifecycle
9. Concurrency, Reliability & Bugs Fixed in Production
10. Anti-Hallucination Architecture
11. Known Limitations & Technical Debt

01

System Diagram



02

Frontend Architecture

React 19 + TypeScript + Vite, two routes (`src/main.tsx`): `/` (upload) and `/audits/:jobId` (results). No global state library — each page owns its own state via `useState` / `useEffect`; there is no shared state to manage beyond what a job-polling loop needs.

Page & component tree

FILE	RESPONSIBILITY
`pages/UploadPage.tsx`	File picker (client-side extension validation), template picker fetched from `GET /templates` with a live item-label preview, or a free-text "Custom" checklist. Passes the `File` object and resolved checklist through `react-router` navigation state to the results page — see §9 for why that's fragile.
`pages/ResultsPage.tsx`	Polls `GET /audits/:jobId` every 2s until a terminal status. Owns the action bar (View/Download Document, Re-run, Export Report, Download PDF) and the document-viewer modal's open/page state.
`components/ChecklistItemCard.tsx`	Pure dispatcher — picks `QueuedRow` / `ErrorRow` / `DoneRow` (in `components/checklist-item/`) by `item.status`. No rendering logic of its own.
`components/checklist-item/DoneRow.tsx`	Verdict badge, statement, contradiction callout, expandable citation list with per-citation page links.
`components/HealthScoreCard.tsx`, `RiskSummaryCard.tsx`	Derived entirely client-side from `job.items` — no extra API call (`lib/scoring.ts`).
`components/DocumentViewerModal.tsx`	In-page PDF preview — see §4 and §9.
`lib/api.ts`	Thin fetch wrappers, one per endpoint. No retry/caching layer.
`lib/types.ts`	Hand-mirrored TypeScript types for every backend response shape. Not generated from the Pydantic schemas — if a backend field changes shape, this file must be updated manually.
`lib/scoring.ts`, `exportReport.ts`, `errors.ts`	Pure functions, no React dependency — health score math, Markdown report generation, raw-error-to-plain-language mapping.

State model

`ResultsPage`'s state is small and entirely derived from one poll loop:

```
const [job, setJob] = useState<Job | null>(null);
const [pollError, setPollError] = useState<string | null>(null);
const [rerunning, setRerunning] = useState(false);
// undefined = viewer closed; null = open with no specific page; N = open at page N
const [viewerPage, setViewerPage] = useState<number | null | undefined>(undefined);
```

Every checklist item's expand/collapse state lives locally inside its own `DoneRow` instance (`useState`), not lifted to `ResultsPage` — deliberately, since no other component needs to read or control it. The one place this mattered: PDF export needs citations visible regardless of their on-screen expand state, solved with CSS rather than lifting state up (see §9).

PDF export: `window.print()`, not a library

The "Download PDF" button calls `window.print()` directly. A `@media print` block in `index.css` hides interactive chrome (action bar, back link, citation show/hide buttons, the viewer modal), forces citation lists visible, and adds `break-inside: avoid` to cards so they don't split awkwardly across a page boundary. No new dependency; the trade-off is the browser's native

print dialog rather than a true one-click download — see the companion project-overview document for the fuller cost/benefit discussion that led here over a client-side PDF library or a server-side Playwright-rendering endpoint.

03

API Layer & Endpoint Reference

`api.py` — FastAPI, one module, no routers/blueprints (small enough not to need the separation yet). Jobs are held in a dict keyed by a generated UUID, `_JOBS: dict`, exactly as before — but every creation and state change is now also written through to a local SQLite file (`src/store.py`, `jobs.db`), and `load_all_jobs()` repopulates `_JOBS` from it at startup. Every route but `/health` also requires `src/auth.py`'s shared API key. Neither is a full production answer — see §11 for exactly what's still missing.

Endpoints

METHOD & PATH	REQUEST	RESPONSE	NOTES
GET `/health`	—	`{"status": "ok"}`	Liveness only — deliberately the one route `require_api_key` exempts, so Docker's `HEALTHCHECK` and platform probes don't need a credential.
GET `/templates`	—	`ChecklistTemplate[]`	Serializes `src/templates.py`'s `TEMPLATES` dict directly.
POST `/audits`	multipart: `file` (required), `template` XOR `checklist` (JSON string array), both optional	`202` `{"job_id", "status": "queued"}`	Validates extension against `SUPPORTED_EXTENSIONS`. Rejects both `template` and `checklist` together as ambiguous. Starts `_run_job` as a detached `asyncio.create_task` — the request returns before any LLM call happens.
GET `/audits/{id}`	—	`Job` (see §7); `report` field present only once `status === "completed"`	Polled by the frontend every 2s. `404` on an unknown id.
GET `/audits/{id}/document`	—	The raw uploaded file bytes, `Content-Disposition: inline`	Added for the Tier 0 document viewer. `content_disposition_type="inline"` is explicit — Starlette's `FileResponse` default is `"attachment"`, which silently forces a download; hit directly in production, fixed. The one route where the API key rides as `?api_key=` instead of a header — opened as a plain `<iframe src>`, which can't attach one.

AUTH ON EVERY ROUTE BUT /HEALTH

`require_api_key` (`src/auth.py`) is a FastAPI dependency on all four routes above except `/health`. It checks an `X-API-Key` header (or the query-param fallback noted above) against the `API_KEY` environment variable. If `API_KEY` is unset, the check is a no-op — every request is allowed. This is a single shared secret, not per-user accounts; see §11.

Job state machine

`queued` → `processing` → `completed` | `failed`. Each item within a job independently moves `queued` → `done` | `error`, throttled by the engine's semaphore (§5) — there is deliberately no `running` state per item, since the API layer doesn't currently track exactly which items are mid-flight versus waiting on the semaphore.

CORS

Explicit dev-only allowlist: `localhost:5173` / `127.0.0.1:5173`. Needs a real allowlist (or a reverse proxy) before this is deployed anywhere the frontend isn't served from those exact origins.

04

Ingestion Layer

Format-specific loaders

FORMAT	LIBRARY	PAGE ACCURACY
`.pdf`	`pypdf`	Exact — one `Document` per real PDF page, free from the format itself.
`.docx`	`python-docx`	Best-effort — Word has no fixed pagination without a render engine. Pages are only detected from explicit manual page breaks (`<w:br w:type="page"/>`); an unmarked file returns one unpaginated `Document` rather than guessing.
`.txt`	Custom regex	Only as accurate as a `[PAGE N]` marker convention the source file must already use; unmarked files also fall back to one unpaginated chunk.

Chunking

```
RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100,
    separators=["\n\n", "\n", ". ", " ", ""]
)
```

`chunk_overlap=100` and the `". "` separator are not defaults — the original values (`chunk_overlap=50`, no sentence separator) let a single load-bearing clause split exactly across a chunk boundary once, and the retriever surfaced only the half without the operative fact. Both changes were verified by reproducing the exact failure and confirming it no longer occurred, four consecutive runs.

Hybrid retriever

`create_hybrid_retriever(documents, k=4)` builds a `BM25Retriever` (sparse, exact keyword/phrase match) and a `Chroma` vector store over local `MiniLMEmbeddings` (dense, semantic match), combined via LangChain's `EnsembleRetriever` at equal 0.5/0.5 weight.

`k` defaults to **4, not 2** — a second, independent fix from the chunking one above. Even with the sentence intact in a single chunk, a narrower top-k retrieval sometimes still failed to surface it, because a different, more keyword-relevant chunk out-ranked it for a given query. Both fixes were required; neither alone resolved the missed-contradiction case that surfaced them.

Local embeddings: `MiniLMEmbeddings`

A hand-rolled `langchain_core.embeddings.Embeddings` subclass wrapping `sentence-transformers/all-MiniLM-L6-v2` (`src/ingestion.py`), replacing an earlier `GoogleGenerativeAIEmbeddings` call. Deliberately *not* built on `langchain-huggingface` — that package pulled in `langchain-core>=1.0` during evaluation, which conflicts with every other pinned LangChain package in `requirements.txt` (all require `<0.4`). The hand-rolled wrapper is the entire integration surface needed and avoids the conflict entirely.

> **WHY LOCAL EMBEDDINGS**
>
> Verified via a controlled, repeated comparison before switching: the same document and checklist, run against `gemini-embedding-001` and against local MiniLM, returned identical verdicts across all four outcome types (a genuine contradiction, a pass, a fail, a not-found) — including the specific contradiction case known to be retrieval-sensitive. No measured quality cost, and it removes ingestion's dependency on network access, an API key, or Gemini's quota entirely.

05

Engine Layer — The Audit Loop

`execute_safe_audit_async(checklist_item, retriever, max_retries=3)` is the unit of work — one checklist question, run to a verdict or an exhausted-retries error. Everything above it (CLI, API job runner) just calls this once per item, concurrently.

Retry loop — two independent failure paths, one recovery mechanism

FAILURE	DETECTED BY	RECOVERY
Malformed structured output (e.g. a string where `page_citations` expects an integer)	`pydantic.ValidationError` raised inside `audit_llm.ainvoke()`	Caught specifically (not a broad `except Exception`, so a genuine API/rate-limit error still fails fast), a corrective note appended to the next prompt, loop continues
Hallucination or altered citation	A separate `run_guardrail_verification` call (§6) flags `is_hallucinated` or `missing_evidence`	The guardrail's `critique_justification` appended to the next prompt, loop continues

Both paths update `last_failure_reason`, so the final exception on exhaustion (`ValueError(f"Failed after {max_retries} attempts. Last issue: {last_failure_reason}")`) reports what actually went wrong on the last attempt — not a generic message, regardless of which failure path caused it.

BUG FIXED — THE VALIDATIONERROR PATH USED TO SKIP THE RETRY LOOP ENTIRELY

Before the `try/except ValidationError` was added, `audit_llm.ainvoke()` had no error handling at all. A single malformed response (the model echoing a `[Source Doc: ..., Page: N]` context tag into `page_citations` instead of extracting the integer) propagated immediately, killing the item with zero retries even though `max_retries=3` existed specifically to give the model another chance. Confirmed via a direct reproduction, then a regression test (`test_retries_after_malformed_structured_output_then_succeeds`).

Concurrency

```
MAX_CONCURRENT_AUDITS = 2

async def run_parallel_due_diligence_with_progress(checklist, retriever, on_item_done=None):
    semaphore = asyncio.Semaphore(MAX_CONCURRENT_AUDITS)
    async def _run_one(index, item):
        async with semaphore:
            try:
                result = await execute_safe_audit_async(item, retriever)
                ...
            except Exception as exc:
                # isolated per-item error -- does not abort the rest of the batch
                ...
    await asyncio.gather(*[_run_one(i, item) for i, item in enumerate(checklist)])
```

Two entrypoints share `execute_safe_audit_async` but differ in failure handling: `run_parallel_due_diligence` (used by the CLI) is all-or-nothing — one item's exhausted retries raises and aborts the whole `asyncio.gather`. `run_parallel_due_diligence_with_progress` (used by the API) catches per-item exceptions and reports them through a callback instead, so one bad item never takes down a job that's otherwise succeeding — this is why the API's per-item `error` status exists at all.

06

Guardrails Layer

`src/guardrails.py` — one function, `run_guardrail_verification(context, generated_output)`, called synchronously inside `asyncio.to_thread` from the engine loop (a second Gemini call, independent of the one that generated the finding being checked).

The prompt, verbatim

```
You are an adversarial Quality Assurance Judge. Your job is to catch
hallucinations, fabrications, or over-extrapolations.
Compare the Evaluated Assertion and its provided Citations against the
Raw Source Text context.

CRITERIA:
1. If the Statement contains ANY fact, date, name, or parameter not
   explicitly found in the Raw Source Text, flag as IS_HALLUCINATED = True.
2. Check if the 'Claimed Verbatim Citations' exist EXACTLY word-for-word
   in the Raw Source Text. If they are modified or missing, flag
   MISSING_EVIDENCE = True.
```

Output is a structured `GuardrailVerdict` (§7), not free text — same schema-enforcement principle as the audit layer. The `critique_justification` field is what feeds back into the next retry attempt's prompt in the engine loop.

> **A REAL EXAMPLE, NOT HYPOTHETICAL**
>
> In production, this guardrail flagged an audit statement claiming a 24-month non-compete term "exceeds the twelve (12) month maximum limit" — correctly, since that 12-month threshold came from the checklist question, not the source document, and criterion 1 above covers exactly that case. Whether that's the guardrail being appropriately strict or overly literal is a legitimate judgment call — documented plainly rather than smoothed over, since it's the clearest illustration of both what this layer catches and its own limitation (it's a second LLM opinion, not a deterministic verifier).

07

Data Models

Two schema families, deliberately kept separate: LLM-facing (what the model is constrained to produce) and orchestration-facing (what the API and frontend pass around). A template's `id` / `label` never enters the LLM-facing schema — the model only ever sees the plain `question` string.

`AuditLog` — src/schemas.py

FIELD	TYPE	NOTES
`verdict`	`Literal["PASSED","FAILED","CONTRADICTORY","NOT_FOUND"]`	Closed set, not a free string.
`checklist_item`	`str`	Set by the engine after generation, not by the model itself.
`raw_statement`	`str`	The finding, in plain language.
`exact_quote_citations`	`List[str]`	Must be verbatim — enforced by the guardrail, not by the schema itself.
`page_citations`	`List[int]`	Paired positionally with citations above. Occasionally comes back a different length than the citations list (an observed LLM-output quirk) — frontend code indexes defensively.
`contradiction_detected`	`bool`	
`contradiction_notes`	`Optional[str]`	Required field, nullable value — must be explicitly present, may be `null`.

`GuardrailVerdict` — src/schemas.py

FIELD	TYPE
`is_hallucinated`	`bool`
`missing_evidence`	`bool`
`critique_justification`	`str`

`ChecklistItem` / `ChecklistTemplate` — src/templates.py

MODEL	FIELDS
`ChecklistItem`	`id: str`, `label: str`, `question: str`
`ChecklistTemplate`	`key: str`, `name: str`, `items: List[ChecklistItem]`

Four templates ship in `TEMPLATES`: `general_contract` (default), `mna_contract`, `financial_report`, `employment_contract`. A custom checklist submitted via the API gets synthetic `ChecklistItem`s with `id="custom-{i}"` and `label` equal to `question` (no separate short form exists for free-typed text).

API job shape (not a Pydantic model — a plain dict, `api.py`)

```
{
  "job_id": str, "status": "queued"|"processing"|"completed"|"failed",
  "document": str, "file_path": str | None, "created_at": str, "error": str | None,
  "items": [{
    "id": str, "label": str, "checklist_item": str,
    "status": "queued"|"done"|"error",
    "result": AuditLog | None, "error": str | None
  }],
  "report": DueDiligenceReport   # present only once status == "completed"
}
```

Frontend types — src/lib/types.ts

Hand-mirrored, not generated — `Job`, `JobItem`, `AuditLog`, `ChecklistTemplate`/`ChecklistItem` match the shapes above field-for-field. There is no shared schema or codegen step between backend and frontend; a backend field change requires a manual, coordinated update to this file.

08

Request Lifecycle

1. `UploadPage` `POST`s to `/audits` with the file and either a `template` key or a JSON `checklist` array.
2. `_resolve_checklist` turns that into a `List[ChecklistItem]`; `_new_job` builds the job dict with every item `queued`; the upload is written to `uploads/{job_id}/{filename}` and its path stored on the job.
3. `asyncio.create_task(_run_job(...))` starts processing in the background — the HTTP response returns immediately with `{job_id, status: "queued"}`, before ingestion or any LLM call happens.
4. `_run_job` loads and chunks the document once, builds one hybrid retriever, then calls `run_parallel_due_diligence_with_progress` with an `on_item_done` callback that mutates the job dict in place as each item resolves.
5. `ResultsPage` polls `GET /audits/{job_id}` every 2 seconds (`POLL_INTERVAL_MS`) and re-renders the full item list on every response — each `ChecklistItemCard` independently reflects its own item's current status.
6. Once every item resolves, the job flips to `completed` (or `failed` on a pipeline-level exception) and the response gains a `report` field — the full `DueDiligenceReport`, ready for the frontend's health score, risk summary, Markdown export, or print-to-PDF, none of which require another API call.

09

Concurrency, Reliability & Bugs Fixed in Production

A running log of the specific defects found and fixed, kept here because each one shaped a piece of the current design — removing the fix without understanding why it exists would likely reintroduce the original bug.

CHUNK BOUNDARY SILENTLY DROPPED A CONTRADICTION

`chunk_overlap=50` let a governing-law clause split exactly across two chunks; the retriever surfaced only the half without "...the laws of the United Kingdom," and the audit reported no contradiction — not because the model reasoned incorrectly, but because it never saw the evidence. Fixed by raising the overlap to 100 and adding a sentence-aware separator. See §4.

TOP-K=2 RETRIEVAL MISSED THE SAME CLAUSE EVEN AFTER THE CHUNK FIX

With the sentence intact in one chunk, a query still sometimes failed to retrieve it because a different chunk ranked higher for that specific question. Fixed by raising `k` to 4 on both the BM25 and vector retrievers. See §4.

VALIDATIONERROR SKIPPED THE RETRY LOOP ENTIRELY

See §5 — a malformed `page_citations` value killed an item on the first attempt, before the 3-attempt retry mechanism ever engaged.

FREE-TIER GEMINI RATE LIMITS, HIT DIRECTLY MORE THAN ONCE

`gemini-2.5-pro` has zero free-tier quota on the key this project runs against. `gemini-flash-latest` allows only 5 requests/minute — too tight once a checklist has more than a couple of items, each worth up to 2 calls per retry attempt. `gemini-flash-lite-latest` had materially more headroom (confirmed empirically — handled 8 rapid calls with zero rate-limit errors) and was adopted for both the audit and guardrail models. `MAX_CONCURRENT_AUDITS=2` exists because a burst of 4 concurrent items still tripped a 429 even on the lite tier.

"VIEW DOCUMENT" DOWNLOADED INSTEAD OF RENDERING INLINE

Starlette's `FileResponse` defaults `content_disposition_type` to `"attachment"`, which forces a download regardless of what the frontend link does. Fixed by passing `content_disposition_type="inline"` explicitly. See §3.

A PLAIN ANCHOR LINK SILENTLY WIPED REACT-ROUTER'S NAVIGATION STATE

The risk-summary panel originally linked to a checklist item via `<a href="#item-id">`. Clicking it triggers the browser's own native fragment navigation, which performs its own `history.pushState` with a fresh state object — discovered to silently overwrite the `File` object "Re-run Audit" depends on, disabling that button with no visible error. Fixed by intercepting the click and calling `scrollIntoView()` directly instead of letting the browser navigate at all.

COLLAPSED CITATIONS WOULD HAVE SILENTLY VANISHED FROM PRINTED PDFS

Citations were conditionally unmounted (`{expanded && (<ul>...)}`) rather than merely hidden, so a print stylesheet had nothing to reveal for any item the user hadn't manually expanded first. Fixed by always rendering the list and toggling visibility with a CSS class instead, which `@media print` can then force open regardless of on-screen state. See §2.

10

Anti-Hallucination Architecture

Four independent mechanisms, layered so no single one is trusted alone:

1. Structural constraint

`AuditLog` requires citations and page numbers as fields, not optional context — structured output enforces this at the schema level, not via a prompt instruction the model could ignore.

2. Independent verification

A second, separate Gemini call re-checks the first one's output against the raw source text (§6) — the model that generated a finding is never the one that grades it.

3. Targeted retry with real feedback

A failed check's specific critique is fed back into the next attempt, not a generic "try again" (§5).

4. Retrieval quality, upstream of generation

A guardrail can only catch a hallucination after it's already been generated. The chunking and retrieval-width fixes in §4/§9 exist because incomplete context is exactly when a model is most likely to fill a gap on its own — fixing retrieval reduces how often the guardrail needs to catch anything in the first place.

VERIFIED, NOT JUST DESIGNED

Running the 12-item M&A template against a document that is actually an investment advisory agreement, nine items correctly returned `NOT_FOUND` rather than fabricating a plausible purchase price or closing date. This is the anti-hallucination framework holding under a template/document mismatch that wasn't even the scenario it was built to handle.

HONEST LIMITATION

The guardrail is itself an LLM call, not a deterministic verifier. It can be wrong in either direction — too lenient, or (see §6's real example) arguably too strict. This reduces hallucination risk meaningfully; it does not eliminate it.

11

Known Limitations & Technical Debt

AREA	LIMITATION	WHY IT'S DELIBERATE, NOT AN OVERSIGHT
Persistence	Jobs are now written through to SQLite and survive a restart, but a job still `queued` / `processing` when the process dies is marked `failed` on the next startup, not resumed; still no worker-process/replica sharing	SQLite solves "survives a restart" without standing up a database server — the right size for a single-instance deployment. Multi-replica sharing needs Postgres/Redis, deferred until multiple instances are actually needed
Auth	One shared `API_KEY` for the whole API — no per-user identity, no way to revoke a single caller without rotating the key for everyone, and unset `API_KEY` means no auth at all	Deliberately minimal: gates the API from the open internet without building accounts before there are multiple distinct users to account for. Real per-user auth is a separate, later decision — see §3
Document viewer	Jumps to the right page; no citation highlighting in place	Highlighting needs page-layout coordinates `pypdf`'s text extraction doesn't capture — a real ingestion-layer rework, scoped as its own project
Item concurrency visibility	API tracks `queued` / `done` / `error` only, no `running` state	Would require the semaphore-holding code to report state transitions the job dict doesn't currently model
Rate limits	Free-tier Gemini quota caps real concurrency and checklist size	Infra/billing decision, not an engineering gap
Frontend/backend schema sync	`lib/types.ts` is hand-mirrored, not generated	No codegen tooling set up yet; small enough surface today to maintain by hand carefully
Uploaded files	Never cleaned up (`uploads/{job_id}/…`)	No retention policy defined yet